CS7GV5 - Real-Time Animation
Student Name: Stephen Komolafe
Student Number: 21336975
YouTube Link: https://youtu.be/lUZ4-2RTvhQ

Assignment #02: Arm Inverse Kinematics

Required Feature: Poseable Mesh in UE5

Mannequin character

A custom C++ actor class APosableCharacter, was implemented using a `UPoseableMeshComponent` to allow direct manipulation of skeletal bones at runtime in C++. Unlike a standard skeletal mesh, which is normally controlled through animation blueprints and the animation thread, the poseable mesh allows bone transforms to be modified directly on the main thread. This is necessary for implementing inverse kinematics manually, as it allows the program to dynamically change bone rotations based on the position of the target.

The poseable mesh component is created and attached to the actor during construction. The Unreal Engine mannequin mesh (SKM_Manny_Simple) is loaded using the ConstructorHelpers utility and assigned to the poseable mesh component. This provides a fully rigged character skeleton that can be manipulated programmatically. The component is attached to a scene root so that it can be positioned correctly within the actor hierarchy.

```
AAPosableCharacter::AAPosableCharacter(){
	PrimaryActorTick.bCanEverTick = true;
	posableMeshComponent_reference = CreateDefaultSubobject<UPoseableMeshComponent>(TEXT("PoseableMesh"));

	posableMeshComponent_reference->SetMobility(EComponentMobility::Movable);
	posableMeshComponent_reference->SetVisibility(true);

	static ConstructorHelpers::FObjectFinder<USkeletalMesh>MannequinMesh(TEXT("/Game/Characters/Mannequins/Meshes/SKM_Manny_Simple"));
	if (MannequinMesh.Succeeded()){
		default_skeletalMesh_reference = MannequinMesh.Object;
		posableMeshComponent_reference->SetSkinnedAssetAndUpdate(default_skeletalMesh_reference);
	}
	(…)
```

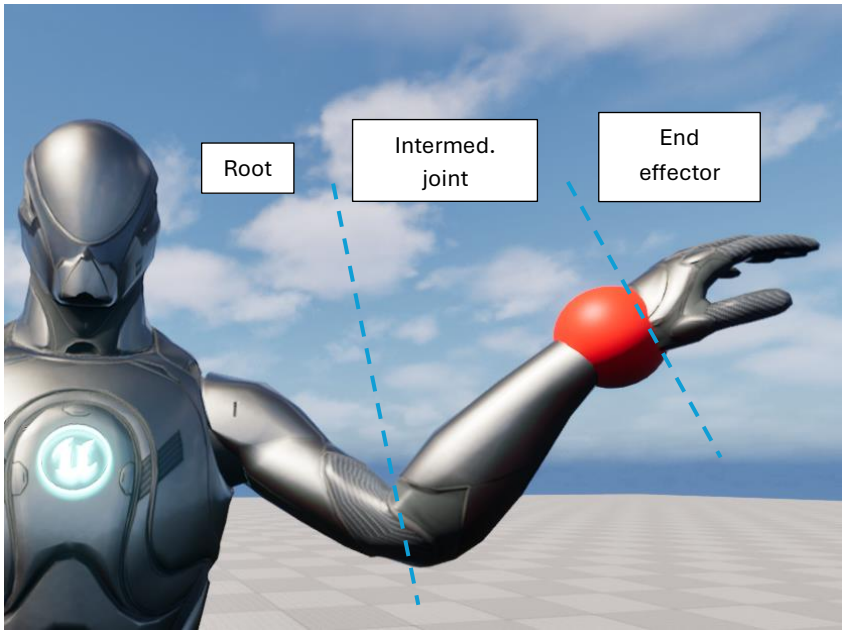
Idle Animation

To make the character feel more natural when idle, a simple breathing animation was implemented as `idle_tickAnimation()` by applying a sinusoidal offset to the spine and clavicle bones every frame in `Tick()`. The original bone rotations are stored at the start of the game (`BeginPlay()`) so that the breathing motion can be applied relative to the base pose. This results in a subtle continuous animation without requiring external animation assets.

```
void AAPosableCharacter::idle_tickAnimation(float DeltaTime){
	if (!posableMeshComponent_reference || !bBasePoseStored)
		return;
	//---spine---
	float Time = GetWorld()->GetTimeSeconds();
	float BreathOffset = FMath::Sin(Time * idle_BreathSpeed) * idle_BreathAmplitude;
	FName SpineBone = "spine_01";
	int32 SpineIndex = posableMeshComponent_reference->GetBoneIndex(SpineBone);

	if (SpineIndex != INDEX_NONE){
		FRotator BaseRot = BasePoseRotations[SpineIndex];
		FRotator NewRot = BaseRot;
		NewRot.Pitch += BreathOffset;
		posableMeshComponent_reference->SetBoneRotationByName(
```

```
                        SpineBone,
                        NewRot,
                        EBoneSpaces::ComponentSpace);
        }

        //---collarbones/(clavicles)---
        float ClavOffset = BreathOffset * 0.5f;
        FName ClavL = "clavicle_l";
        int32 ClavLIndex = posableMeshComponent_reference->GetBoneIndex(ClavL);
        if (ClavLIndex != INDEX_NONE){
                FRotator BaseRot = BasePoseRotations[ClavLIndex];
                FRotator NewRot = BaseRot;
                NewRot.Roll += ClavOffset;

                posableMeshComponent_reference->SetBoneRotationByName(
                        ClavL,
                        NewRot,
                        EBoneSpaces::ComponentSpace);
        }

        FName ClavR = "clavicle_r";
        int32 ClavRIndex = posableMeshComponent_reference->GetBoneIndex(ClavR);
        if (ClavRIndex != INDEX_NONE){
                FRotator BaseRot = BasePoseRotations[ClavRIndex];
                FRotator NewRot = BaseRot;
                NewRot.Roll -= ClavOffset;
                posableMeshComponent_reference->SetBoneRotationByName(
                        ClavR,
                        NewRot,
                        EBoneSpaces::ComponentSpace);
        }
        posableMeshComponent_reference->RefreshBoneTransforms();
}
```

Required Feature: Multi-bone IK in 3D



The inverse kinematics system was implemented using the FABRIK (Forward And Backward Reaching Inverse Kinematics) algorithm to control a three-bone chain consisting of the upper arm (root), lower arm (intermediate joint), and hand (end effector). The goal of the solver is to adjust the joint positions so that the hand bone reaches a specified target location represented by a sphere in the scene. The solver operates in component space to maintain consistency with the poseable mesh transformations.

The algorithm retrieves and stores the current positions of the upper arm, lower arm, and hand bones to form the joints of the IK chain. Then the lengths of the upper arm and lower arm segments are calculated and used as constraints so the arm can maintain its natural proportions. If the target position lies outside the maximum reach of the arm, the solver handles the unreachable case by stretching the chain toward the target direction while preserving bone lengths.

```
FVector UpperPos = posableMeshComponent_reference
        ->GetBoneLocationByName(IK_UpperArm, EBoneSpaces::ComponentSpace);

FVector LowerPos = posableMeshComponent_reference
        ->GetBoneLocationByName(IK_LowerArm, EBoneSpaces::ComponentSpace);

FVector HandPos = posableMeshComponent_reference
        ->GetBoneLocationByName(IK_Hand, EBoneSpaces::ComponentSpace);
```

For reachable targets, the solver performs several FABRIK iterations consisting of backward and forward passes. In the backward stage the end effector is placed at the target, and the joints are adjusted, moving toward the root. In the forward stage the root position is restored, and the chain is reconstructed toward the end effector. The iteration stops when the hand is within a defined tolerance distance from the target or when a maximum number of iterations is reached.

```
for (int32 Iter = 0; Iter < IK_MaxIterations; Iter++){
// BACKWARD
      Joints[2] = TargetPos;

      FVector Dir = (Joints[1] - Joints[2]).GetSafeNormal();
      Joints[1] = Joints[2] + Dir * LengthLower;

      FVector Dir = (Joints[0] - Joints[1]).GetSafeNormal();
      Joints[0] = Joints[1] + Dir * LengthUpper;

// FORWARD
      Joints[0] = RootPosition;

      FVector Dir = (Joints[1] - Joints[0]).GetSafeNormal();
      Joints[1] = Joints[0] + Dir * LengthUpper;

      FVector Dir = (Joints[2] - Joints[1]).GetSafeNormal();
      Joints[2] = Joints[1] + Dir * LengthLower;

      if (FVector::Distance(Joints[2], TargetPos) < IK_Tolerance)
          break;
}
```

Required Feature: Scripted Animation

Scripted animation for the end effector was implemented in C++ using a `USplineComponent` attached to the actor. The spline sets a series of key points that forms the path that the hand target should follow over time. These points are added during the `BeginPlay()` function and form a smooth trajectory in three-dimensional space. The IK target sphere moves along this path, and the inverse kinematics solver continuously adjusts the arm joints so that the hand follows the moving target.

```
        HandSpline->ClearSplinePoints();

        HandSpline->AddSplinePoint(FVector(60, -30, 120), ESplineCoordinateSpace::Local);
        HandSpline->AddSplinePoint(FVector(80, -10, 150), ESplineCoordinateSpace::Local);
        HandSpline->AddSplinePoint(FVector(60, 30, 130), ESplineCoordinateSpace::Local);
        HandSpline->AddSplinePoint(FVector(60, 0, 110), ESplineCoordinateSpace::Local);

        HandSpline->UpdateSpline();
```

The animation progresses using a time parameter that advances each frame in `Tick()`. This value is normalised to produce an interpolation parameter, `Alpha`, between 0 and 1, which represents the current position along the spline. The corresponding location along the spline is calculated using the distance along the curve and then applied to the target sphere's relative location. As the sphere moves, the IK solver automatically updates the arm pose to reach the new position.

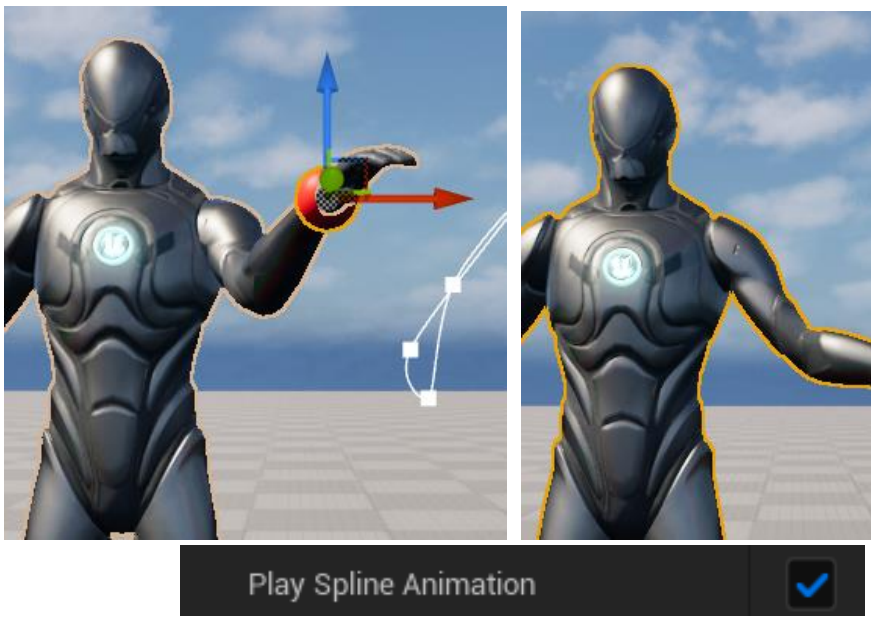
```
void AAPosableCharacter::UpdateSplineAnimation(float DeltaTime){
        if (!bPlaySplineAnimation || !HandSpline || !targetSphere)
                return;

        SplineTime += DeltaTime;
        float Alpha = FMath::Fmod(SplineTime, SplineDuration) / SplineDuration;
        float EasedAlpha = EaseInOut(Alpha);
        float SplineLength = HandSpline->GetSplineLength();
        float Distance = EasedAlpha * SplineLength;
        FVector NewLocation = HandSpline->GetLocationAtDistanceAlongSpline(
                Distance,
                ESplineCoordinateSpace::Local);

        targetSphere->SetRelativeLocation(NewLocation);
}
```

To improve the visual quality of the motion, an ease-in/ease-out curve was applied using a smoothstep function. Using this function, the animation begins and ends smoothly instead of moving at a constant speed. As a result, the hand movement appears more natural and visually appealing when following the spline path.

```
float EaseInOut(float Alpha){
        return Alpha * Alpha * (3.0f - 2.0f * Alpha);
}
```

Required Feature: Advanced Features

1. Manual/Automated IK Toggle



Several additional features were implemented to improve the realism and usability of the inverse kinematics system. One of these features is the ability to toggle between manual target control and scripted spline animation. The function, `ToggleSplineAnimation()`, allows the user to switch between manual target positioning via the editor and automatic spline animation where the hand follows a predefined spline path, allowing demonstration of both interactive IK and automated animation modes. This functionality is implemented through a boolean variable that determines whether the spline animation should be updated during each frame.

```
void AAPosableCharacter::ToggleSplineAnimation(){
        bPlaySplineAnimation =! bPlaySplineAnimation;

        if (!bPlaySplineAnimation){
                UE_LOG(LogTemp, Warning, TEXT("Manual Target Mode"));
        }
        else{
                UE_LOG(LogTemp, Warning, TEXT("Spline Animation Mode"));
        }
}
```

And in `Tick()`:

```
        if (bPlaySplineAnimation){
                UpdateSplineAnimation(DeltaTime);
        }

        idle_tickAnimation(DeltaTime);
        SolveLeftArmIK();
```

2.Unreachable Target Handling

The IK system detects when the target position exceeds the total arm length, referred to as an unreachable position. In this case, the solver does not attempt iterative correction. Instead, the arm is fully extended in the direction of the target while preserving bone lengths. To better demonstrate reachable and unreachable positions, a red debug sphere was added and is rendered anytime the target sphere is placed at an unreachable location.

```
float TotalLength = LengthUpper + LengthLower;
float RootToTargetDist = FVector::Distance(UpperPos, TargetPos);

if (RootToTargetDist > TotalLength){
        FVector Direction = (TargetPos - UpperPos).GetSafeNormal();

        Joints[1] = UpperPos + Direction * LengthUpper;
        Joints[2] = Joints[1] + Direction * LengthLower;

        if (bDrawDebugInfo){
                DrawDebugSphere(GetWorld(), TargetWorld, 12, 12, FColor::Red, false, 0.0f);
        }
}
```

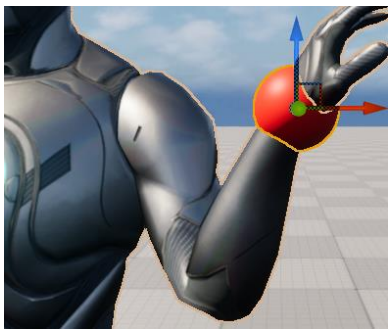
2. Pole Vector Constraint (Controlled Elbow Direction)

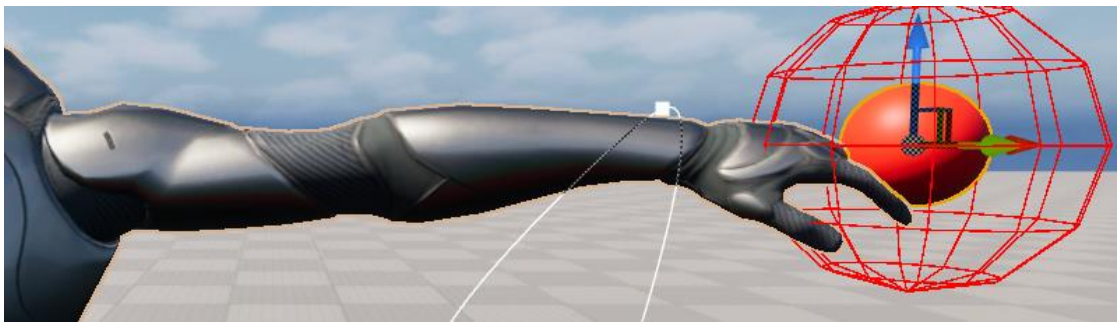
A pole vector constraint was implemented to control the bending direction of the elbow. Without this constraint, the FABRIK solver may produce ambiguous bending planes and cause unnatural flipping behaviour. The pole vector defines a plane in which the elbow

must lie. The elbow joint is projected onto this plane after the FABRIK iterations, ensuring stable and anatomically correct bending.

```
FVector PoleWorld = GetActorTransform().TransformPosition(ElbowPoleOffset);
FVector Pole = posableMeshComponent_reference
->GetComponentTransform()
.InverseTransformPosition(PoleWorld);

FVector RootToHand = (Joints[2] - Joints[0]).GetSafeNormal();
FVector RootToPole = (Pole - Joints[0]).GetSafeNormal();

FVector PlaneNormal = FVector::CrossProduct(RootToHand, RootToPole).GetSafeNormal();

FVector RootToElbow = Joints[1] - Joints[0];
float ProjectionLength = FVector::DotProduct(RootToElbow, RootToHand);

FVector ProjectedPoint = Joints[0] + RootToHand * ProjectionLength;

float ElbowDistance = FVector::Distance(ProjectedPoint, Joints[1]);

FVector CorrectDir = FVector::CrossProduct(PlaneNormal, RootToHand).GetSafeNormal();
Joints[1] = ProjectedPoint + CorrectDir * ElbowDistance;
```

3. Anatomical Joint Limits





To prevent hyperextension and unnatural deformation, anatomical joint limits were implemented for the elbow. A pole vector constraint is used to guide the elbow direction so that the arm bends in a realistic plane instead of twisting unpredictably. Furthermore, the elbow joint angle is clamped to a realistic human range between approximately 5° and 150° to prevent unnatural hyperextension. If the computed angle exceeds the limits, a corrective quaternion is applied to rotate the lower arm back within range.

```
FVector UpperDir = (Joints[1] - Joints[0]).GetSafeNormal();
FVector LowerDir = (Joints[2] - Joints[1]).GetSafeNormal();

float Angle = FMath::RadiansToDegrees(
        FMath::Acos(FVector::DotProduct(UpperDir, LowerDir))
);

float MinAngle = 5.0f;
float MaxAngle = 150.0f;

if (Angle < MinAngle || Angle > MaxAngle){
        float ClampedAngle = FMath::Clamp(Angle, MinAngle, MaxAngle);
        float AngleDiff = ClampedAngle - Angle;

        FVector Axis = FVector::CrossProduct(UpperDir, LowerDir).GetSafeNormal();
        FQuat CorrectionQuat(Axis, FMath::DegreesToRadians(AngleDiff));

        LowerDir = CorrectionQuat.RotateVector(LowerDir);
        Joints[2] = Joints[1] + LowerDir * LengthLower;
}
```